

Classification in Machine Learning

Complete Course - All Models

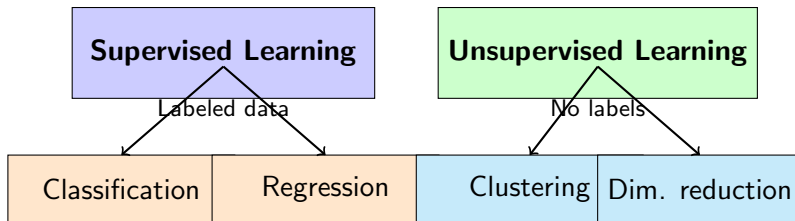
Meyssem Bouzaïen

Academic Year: 2025 - 2026

Course Outline

- 1 What is Classification?
- 2 K-Nearest Neighbors (KNN)
- 3 Decision Trees
- 4 Logistic Regression
- 5 Support Vector Machines (SVM)
- 6 Random Forests
- 7 Evaluation Metrics
- 8 Model Comparison
- 9 Putting It Into Practice
- 10 Practice Exercises
- 11 Best Practices and Conclusion

Reminder: Types of Learning



Today

We focus on **Classification**

What is classification?

Classification consists of **predicting a category** (class) for new observations, based on labeled examples.

Input (Features):

- Measurable characteristics
- Descriptive variables
- Example: size, weight, color...

Output (Class):

- Predicted category
- Discrete variable
- Example: cat, dog, bird

Classification Examples

Application	Features (X)	Classes (y)
Email spam	Words, sender, links	Spam / Not spam
Medical diagnosis	Symptoms, test results	Sick / Healthy
Image recognition	Image pixels	Cat / Dog / Bird
Bank credit	Income, age, history	Approved / Denied
Sentiment analysis	Text of a review	Positive / Negative / Neutral
Flower recognition	Sepal/petal size	Setosa / Versicolor / Virginica

Classification is everywhere!

Binary vs Multi-class Classification

Binary Classification

Only 2 classes

- Yes / No
- Spam / Ham (not spam)
- Sick / Healthy
- Positive / Negative

Multi-class Classification

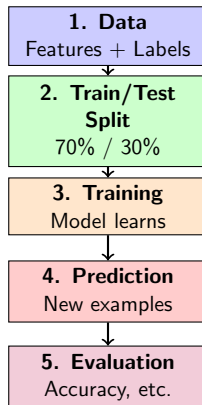
More than 2 classes

- 3 types of Iris flowers
- 10 digits (0-9)
- 26 letters (A-Z)
- 1000 object categories

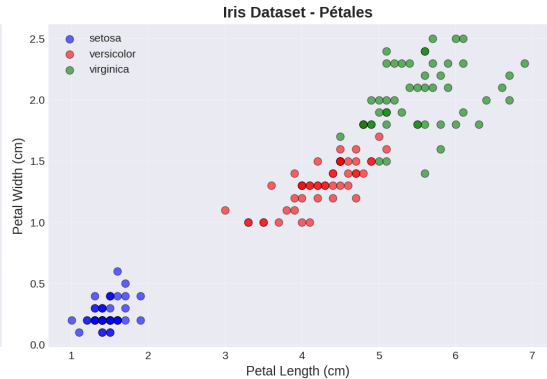
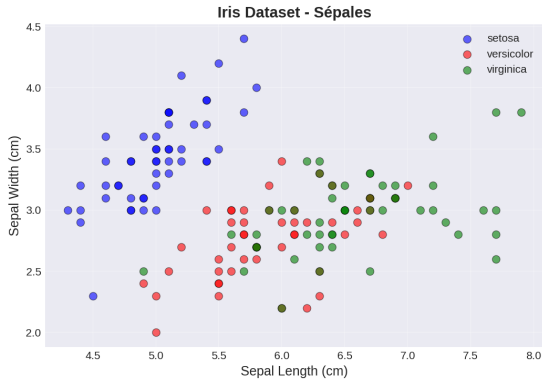
Important

All of today's algorithms work for both!

The Classification Process



Iris Dataset - Our Example



150 flowers, 3 species, 4 features (sepals and petals)

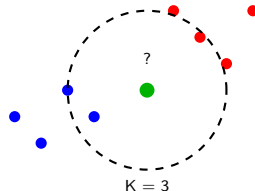
KNN: The Simplest Algorithm

Core Idea

“Tell me who your neighbors are, and I’ll tell you who you are”

Principle:

- 1 Find the K nearest neighbors
- 2 Look at their class
- 3 Majority vote
- 4 Assign the winning class



KNN: How Does It Work? (1/2)

Step 1: Compute the Distance

For each point in the dataset, compute its distance to the new point.

Euclidean distance (most common):

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Example

Point A = (1, 2) and Point B = (4, 6)

$$d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

KNN: How Does It Work? (2/2)

Step 2: Sort by Distance

Keep the K closest points

Step 3: Majority Vote

Count the classes among the K neighbors

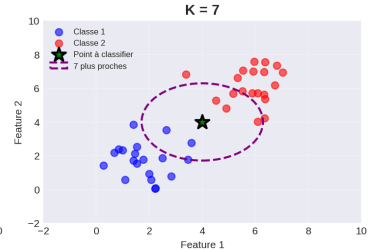
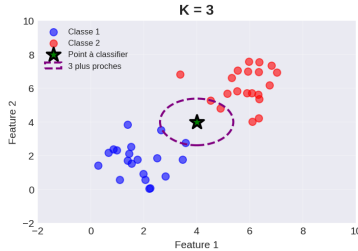
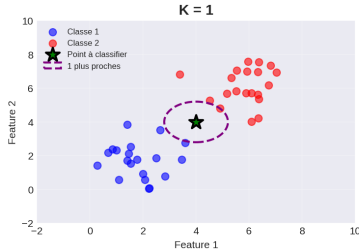
Example with $K=5$

- Neighbor 1: Class A (distance: 1.2)
- Neighbor 2: Class A (distance: 1.5)
- Neighbor 3: Class B (distance: 1.8)
- Neighbor 4: Class A (distance: 2.1)
- Neighbor 5: Class B (distance: 2.3)

Vote: A=3, B=2 → Predicted class = A

The Parameter K: Impact

K-Nearest Neighbors: Impact du paramètre K



Advice

- K too small → overfitting
- K too large → underfitting
- Common values: $K = 3, 5, 7$

KNN with Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load the data
iris = load_iris()
X, y = iris.data, iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Create the KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# 4. Train
knn.fit(X_train, y_train)

# 5. Predict
y_pred = knn.predict(X_test)

# 6. Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

KNN: Advantages and Disadvantages

Advantages

- Simple to understand
- No real training step
- Naturally multi-class
- No assumptions

Disadvantages

- Slow (large datasets)
- Sensitive to scale
- Sensitive to noise
- Memory-intensive

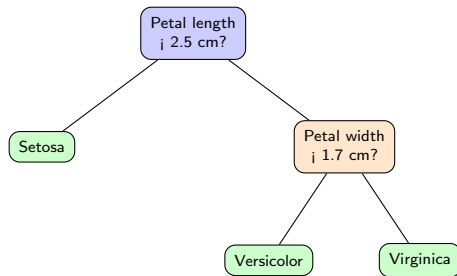
Important

Always normalize the data for KNN!

Decision Trees: Intuition

The Idea

A decision tree asks a series of **questions** about the features to arrive at a decision.



Like a decision flowchart!

How Do We Build a Tree?

Algorithm

- 1 Choose the best question
- 2 Split the data
- 3 Repeat for each branch
- 4 Stop when all examples belong to the same class

Key Question

How do we choose the **best** question?

→ The one that best separates the classes!

Gini Index (Simplified)

Principle

We want **pure** groups (a single class per group)

$$Gini = 1 - \sum_i p_i^2$$

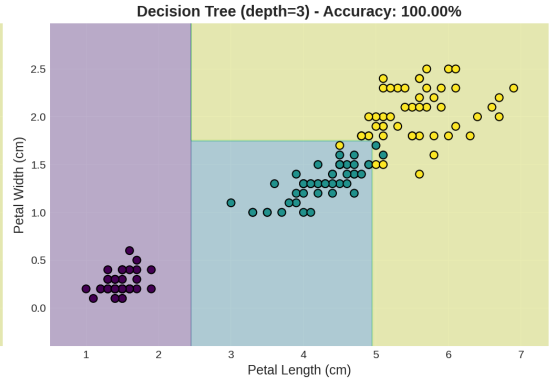
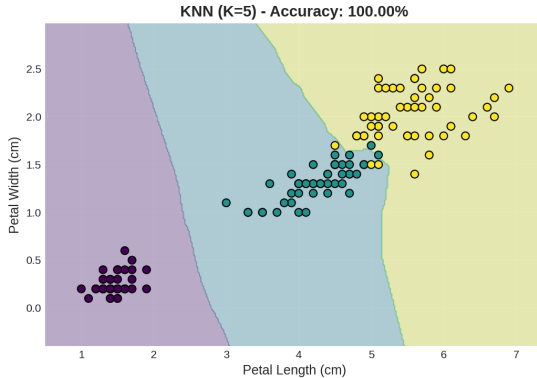
- $Gini = 0 \rightarrow$ perfectly pure (excellent!)
- $Gini = 0.5 \rightarrow$ 50/50 mix (bad)

Simple Example

Group A: 10 blue, 0 red $\rightarrow Gini = 0$ (pure!)

Group B: 5 blue, 5 red $\rightarrow Gini = 0.5$ (impure)

Decision Boundaries: KNN vs Tree



KNN: Smooth boundaries — **Tree:** Rectangular boundaries

Decision Trees with Scikit-learn

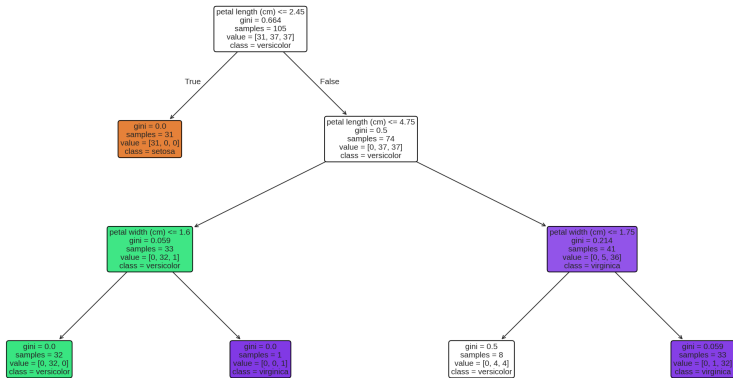
```
1 from sklearn.tree import DecisionTreeClassifier
2
3 # Create the tree
4 tree = DecisionTreeClassifier(
5     max_depth=3,          # Max depth
6     min_samples_split=2  # Min to split
7 )
8
9 # Train
10 tree.fit(X_train, y_train)
11
12 # Predict
13 y_pred = tree.predict(X_test)
14
15 # Evaluate
16 accuracy = accuracy_score(y_test, y_pred)
17 print(f"Accuracy: {accuracy*100:.2f}%")
```

Controlling Complexity

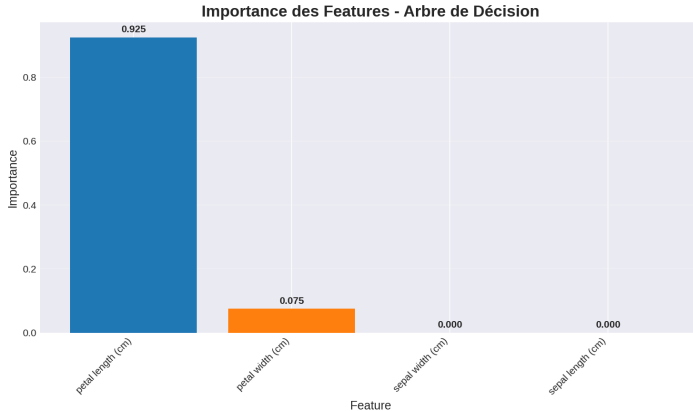
- **max_depth**: Maximum depth
 - Too large → overfitting
 - Too small → underfitting
- **min_samples_split**: Minimum to split
- **min_samples_leaf**: Minimum in a leaf

The Full Tree Visualized

Arbre de Décision - Dataset Iris (4 features)



Feature Importance



Trees show which features matter most!

Trees: Advantages and Disadvantages

Advantages

- Easy to visualize
- No normalization
- Fast
- Interpretable
- Feature importance

Disadvantages

- Overfitting
- Unstable
- Rigid boundaries
- Less accurate

Solution

Random Forests → More robust!

Logistic Regression: Principle

Careful!

Despite its name, this is a **CLASSIFICATION** algorithm

Principle

Computes the **probability** that an example belongs to a class

Characteristics:

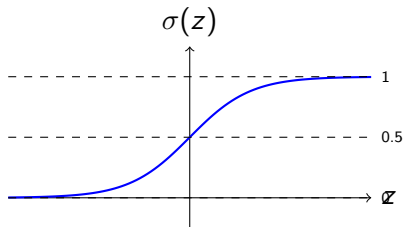
- Excellent for binary classification
- Multi-class extension possible
- Provides probabilities (0 to 1)
- Fast and efficient

Sigmoid Function

Transformation

Transforms any value into a probability (0 to 1)

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



How Does It Work?

- 1 Linear combination of features: $z = w_1x_1 + w_2x_2 + \dots + b$
- 2 Apply sigmoid: $p = \sigma(z)$
- 3 Decision: If $p > 0.5 \rightarrow$ Class 1, otherwise Class 0

Example

Features: [height=1.70, weight=70]

Learned weights: $w_1 = 0.5, w_2 = 0.3, b = -40$

$$z = 0.5 \times 1.70 + 0.3 \times 70 - 40 = 1.85$$

$$p = \sigma(1.85) = 0.86 = 86\%$$

$\rightarrow$ Class 1 with 86% confidence

Logistic Regression: Code

```
from sklearn.linear_model import LogisticRegression

# Create the model
log_reg = LogisticRegression(max_iter=1000)

# Train
log_reg.fit(X_train, y_train)

# Predict
y_pred = log_reg.predict(X_test)

# Probabilities
y_proba = log_reg.predict_proba(X_test)
print(y_proba[0]) # Example: [0.05, 0.70, 0.25]

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Logistic Regression: Advantages

Advantages

- Very fast
- Provides probabilities
- Good for binary
- Few parameters
- Interpretable

Disadvantages

- Linear only
- Weaker if non-linear
- Needs normalization

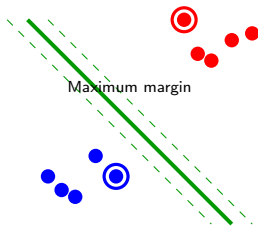
When to Use It?

Simple binary classification, need for probabilities, speed

SVM: Principle

The Idea

Find the **best boundary** that separates the classes with the widest margin



Goal: Maximize the distance between the classes

Definition

Support vectors are the points closest to the boundary

- These are the most important points
- Only these points influence the boundary
- Removing the other points changes nothing!

Why SVM?

Support Vector Machine

→ Named after these critical points

The Kernel Trick

Problem

What if the data isn't linearly separable?

Solution: Kernels

Transform the data into a higher-dimensional space

Popular kernels:

- **linear**: For linear data (fast)
- **rbf** (radial): For non-linear data (default)
- **poly**: Polynomial
- **sigmoid**: Like a neural network

SVM: Python Code

```
from sklearn.svm import SVC

# Create the SVM model
svm = SVC(
    kernel='rbf',      # Kernel (rbf, linear, poly)
    C=1.0,             # Regularization
    gamma='scale'      # Kernel coefficient
)

# Train
svm.fit(X_train, y_train)

# Predict
y_pred = svm.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```


Important Parameters

- **C**: Controls the margin/error trade-off
 - Small C $\rightarrow$ wide margin, more errors (underfitting)
 - Large C $\rightarrow$ narrow margin, fewer errors (overfitting)
- **kernel**: Type of transformation
- **gamma**: For rbf/poly kernels
 - Small gamma $\rightarrow$ wide influence
 - Large gamma $\rightarrow$ local influence (overfitting)

SVM: Advantages and Disadvantages

Advantages

- Very effective
- Good in high dimensions
- Versatile (kernels)
- Robust

Disadvantages

- Slow (large datasets)
- Sensitive to parameters
- Hard to interpret
- No direct probabilities

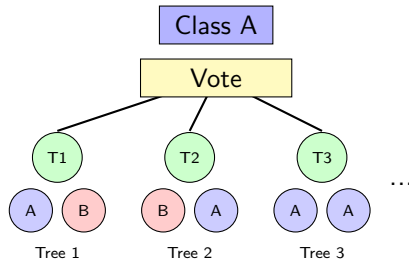
When to Use It?

Medium-sized datasets, high dimension, complex boundaries

Random Forests: Principle

The Idea

Combine **several trees** to be more robust and accurate



Principle: Many trees vote together

How Does It Work?

Algorithm

- ① Create N different trees
- ② Each tree is trained on:
 - A random subset of the data (bootstrap)
 - A random subset of the features
- ③ To predict: each tree votes
- ④ Final class = majority vote

Why Does This Work?

- Reduces variance (overfitting)
- “Wisdom of the crowd”
- Errors cancel each other out

Random Forests: Code

```
from sklearn.ensemble import RandomForestClassifier

# Create the forest
rf = RandomForestClassifier(
    n_estimators=100,      # Number of trees
    max_depth=10,         # Max depth
    min_samples_split=2,
    random_state=42
)

# Train
rf.fit(X_train, y_train)

# Predict
y_pred = rf.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Important Parameters

- **n_estimators**: Number of trees
 - More → better, but slower
 - Typical: 100-500
- **max_depth**: Depth of the trees
- **max_features**: Number of features per tree
 - 'sqrt' → square root (default)
 - 'log2' → base-2 logarithm
- **min_samples_split/leaf**: Controls complexity

Random Forests: Advantages

Advantages

- Very high performance
- Resistant to overfitting
- Handles missing data well
- Shows feature importance
- Works well "out of the box"
- Handles high dimensions
- Parallelizable

Disadvantages

- Less interpretable than a single tree
- Slower than a single tree
- More memory

When to Use Random Forests?

Excellent choice when:

- You need good performance
- Medium/large dataset
- Extreme interpretability isn't required
- You need robustness
- First approach to a new problem

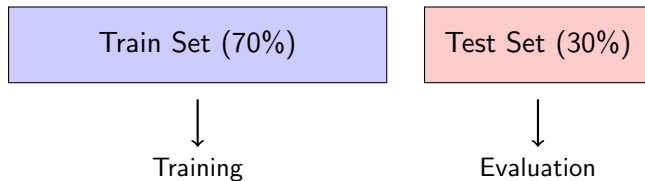
Advice

Random Forests is often the **best starting point** for a classification problem!

Train/Test Split

Golden Rule

NEVER evaluate on the training data!



Common proportions:

- 70/30, 80/20, 75/25

Accuracy

Definition

Percentage of correct predictions

$$Accuracy = \frac{\text{Correct predictions}}{\text{Total}}$$

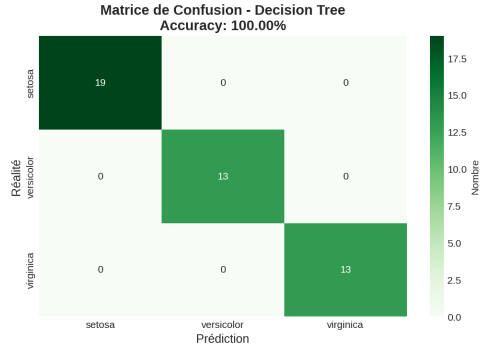
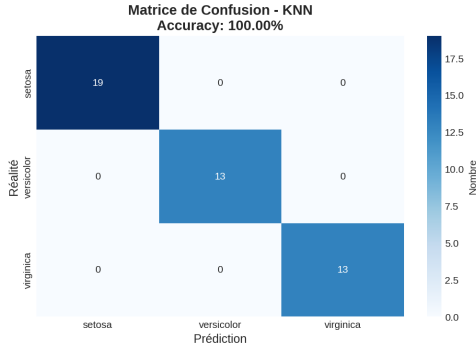
Example

100 predictions, 95 correct $\rightarrow$ Accuracy = 95%

Warning!

Can be misleading if classes are imbalanced!

Confusion Matrix



Visualizes all predictions vs actual values

Precision, Recall, F1-Score

Precision

$$\frac{TP}{TP+FP}$$

Among positive predictions, how many are correct?

Recall

$$\frac{TP}{TP+FN}$$

Among actual positives, how many detected?

F1-Score

$$2 \times \frac{P \times R}{P + R}$$

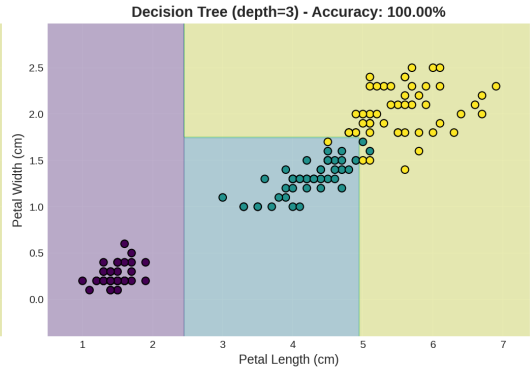
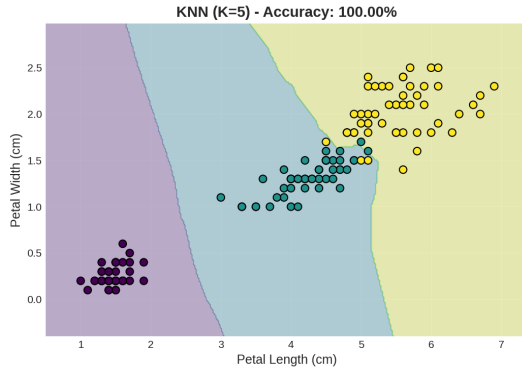
Harmonic mean

Usefulness

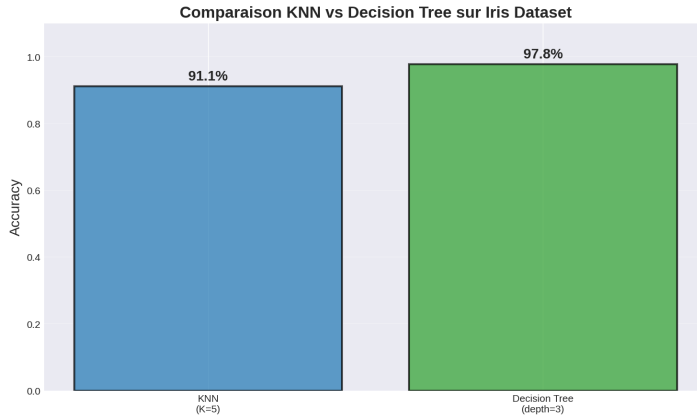
Balances precision/recall

Essential for imbalanced classes

Decision Boundaries



Comparative Performance



Which Model to Choose?

Situation	Model
Small dataset, need simplicity	KNN
Need interpretability	Trees
Simple binary classification	Log. Reg.
High dimension, non-linear	SVM
Best overall performance	Random Forest

Advice

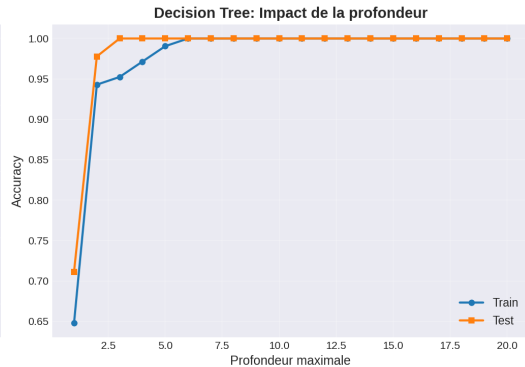
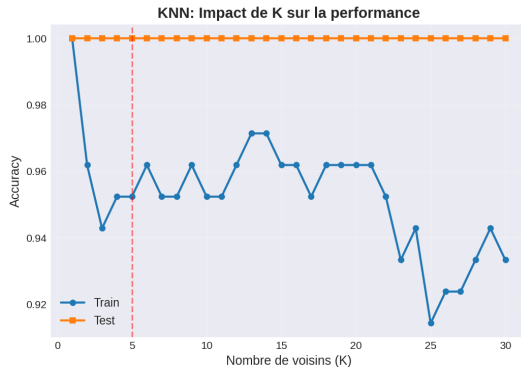
Try several models and compare!

Summary Table

	KNN	Trees	Log Reg	SVM	RF
Simplicity	+++	+++	++	+	++
Performance	++	++	++	+++	+++
Speed	+	+++	+++	+	++
Interpretable	++	+++	++	+	+
Large datasets	+	++	+++	+	++

+ = weak, ++ = medium, +++ = excellent

Avoiding Overfitting



Rule

Monitor Train vs Test accuracy

Anti-Overfitting Strategies

For KNN

- Increase K
- Normalize data
- Reduce features

For Trees/RF

- Limit max_depth
- Increase min_samples_split
- Pruning

For all

- More data
- Cross-validation
- Regularization

Complete Example: Comparing All Models

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# Create the models
models = {
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Tree': DecisionTreeClassifier(max_depth=3),
    'LogReg': LogisticRegression(max_iter=1000),
    'SVM': SVC(kernel='rbf'),
    'RF': RandomForestClassifier(n_estimators=100)
}
```

Complete Example (continued)

```
# Train and evaluate each model
results = {}
for name, model in models.items():
    # Train
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Evaluate
    acc = accuracy_score(y_test, y_pred)
    results[name] = acc

    print(f"{name}: {acc*100:.2f}%")

# Best model
best = max(results, key=results.get)
print(f"\nBest: {best} ({results[best]*100:.2f}%")
```

Recommended Workflow

- 1 **Explore** the data
- 2 **Prepare**: cleaning, normalization
- 3 **Split** Train/Test
- 4 **Test** several simple models
- 5 **Compare** performance
- 6 **Optimize** the best one
- 7 **Validate** on the test set
- 8 **Analyze** errors

Important

Never optimize on the test set!

Normalization: When and How?

Models that NEED it:

- KNN (distances!)
- SVM
- Logistic Regression

Models that DON'T need it:

- Decision Trees
- Random Forests

StandardScaler method:

- Mean = 0
- Standard deviation = 1

Normalization: Code

```
1 from sklearn.preprocessing import StandardScaler
2
3 # Create the scaler
4 scaler = StandardScaler()
5
6 # IMPORTANT: fit on train only!
7 scaler.fit(X_train)
8
9 # Transform train and test
10 X_train_scaled = scaler.transform(X_train)
11 X_test_scaled = scaler.transform(X_test)
12
13 # Use the normalized data
14 knn = KNeighborsClassifier(n_neighbors=5)
15 knn.fit(X_train_scaled, y_train)
16 y_pred = knn.predict(X_test_scaled)
```

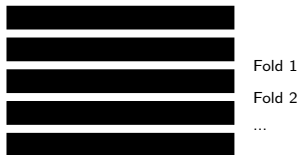
Cross-Validation

Problem

A single train/test split can be lucky or unlucky

Solution: K-Fold Cross-Validation

- 1 Split the data into K parts
- 2 For each part:
 - Use it as the test set
 - The rest = train
- 3 Average the K results



Cross-Validation: Code

```
from sklearn.model_selection import cross_val_score

# Model
model = RandomForestClassifier(n_estimators=100)

# Cross-validation with 5 folds
scores = cross_val_score(
    model, X, y,
    cv=5,          # 5 folds
    scoring='accuracy'
)

print(f"Scores: {scores}")
print(f"Mean: {scores.mean():.2f}")
print(f"Std dev: {scores.std():.2f}")
```

Advantages:

- More robust estimate
- Uses all the data
- Detects instability

Exercise 1: Wine Dataset

Goal

Classify 3 types of wine based on their chemical characteristics

To do:

- 1 Load: `from sklearn.datasets import load_wine`
- 2 Explore: shape, features, classes
- 3 Train/Test split (70/30)
- 4 Test KNN, Tree, Random Forest
- 5 Compare accuracies
- 6 Display confusion matrices
- 7 Identify the best model

Exercise 2: Optimization

Goal

Find the best hyperparameters

To do:

- 1 KNN: test K from 1 to 20
- 2 Create accuracy vs K plot
- 3 Trees: test max_depth from 1 to 10
- 4 Random Forest: test n_estimators
- 5 Identify optimal values
- 6 Compare with default parameters

Best Practices Checklist

- 1 Always split Train/Test BEFORE anything else
- 2 Normalize when needed (KNN, SVM, LogReg)
- 3 NEVER touch the test set during development
- 4 Test several models
- 5 Use several metrics (not just accuracy)
- 6 Check for overfitting (train vs test)
- 7 Cross-validation for robustness
- 8 Visualize results (confusion matrix)
- 9 Document code and choices
- 10 Understand the model's errors

When to Use Each Model?

- **KNN**: Small dataset, few features, quick prototype
- **Trees**: Need interpretability, mixed features
- **Log. Reg.**: Simple binary, need for probabilities
- **SVM**: High dimension, complex boundaries
- **Random Forests**: Starting point, need for performance

Recap: 5 Algorithms

You have learned:

- ① **KNN**: Vote of the K neighbors
- ② **Trees**: Successive questions
- ③ **Logistic Regression**: Probabilities via sigmoid
- ④ **SVM**: Maximum margin
- ⑤ **Random Forests**: Ensemble of trees

Essential Concepts:

- Train/Test split
- Metrics: Accuracy, Precision, Recall, F1
- Overfitting vs Underfitting
- Normalization
- Cross-validation

Resources to Go Further

Documentation

- Scikit-learn: `scikit-learn.org`
- Andrew Ng's course: Machine Learning (Coursera)
- Book: "Hands-On ML" (Géron)

Datasets to Practice

- Scikit-learn: `load_iris`, `load_wine`, `load_breast_cancer`
- Kaggle: `kaggle.com/datasets`
- UCI ML Repository